
4D-IA3 :

Apprentissage Automatique 2

Secil ERCAN

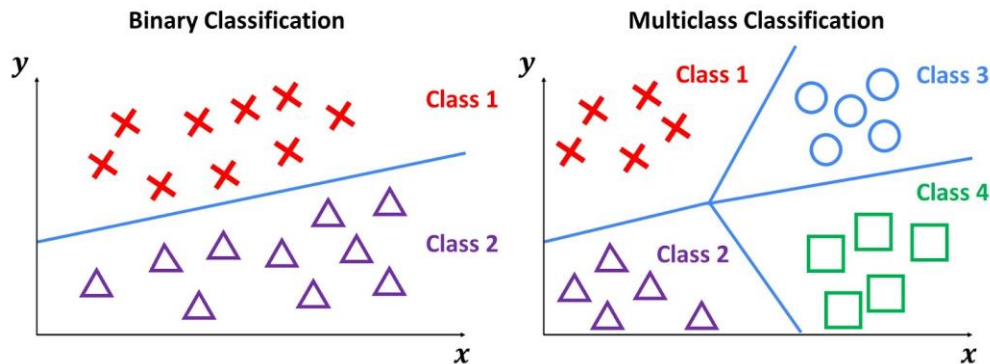
secil.ercan@esiee.fr

Bureau : 5307

Classification

Classes

- **Sortie discrète**
- Classes supposées **mutuellement exclusives**
- Classification **binaire vs Multi-classe**
- L'ensemble des classes est supposé **couvrir tout l'espace des sorties possibles**

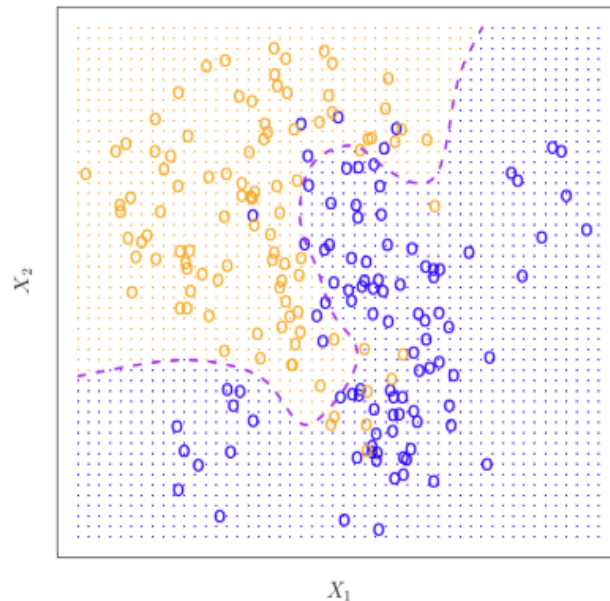


Classification

kNN

Régression Logistique & Vers kNN

- **Hypothèse** : La relation non linéaire et complexe/irrégulière
- **Régression logistique** n'est pas approprié pour ces relations
- **kNN** peut gérer les relations non linéaires et complexes en classification (comme en régression)



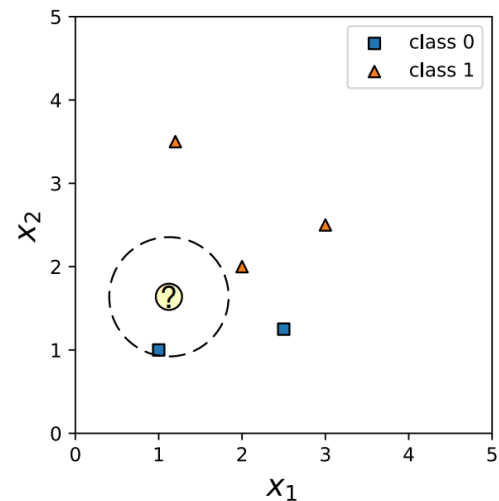
Procedure

Le début du procédure de kNN pour la classification est le même celui pour la régression

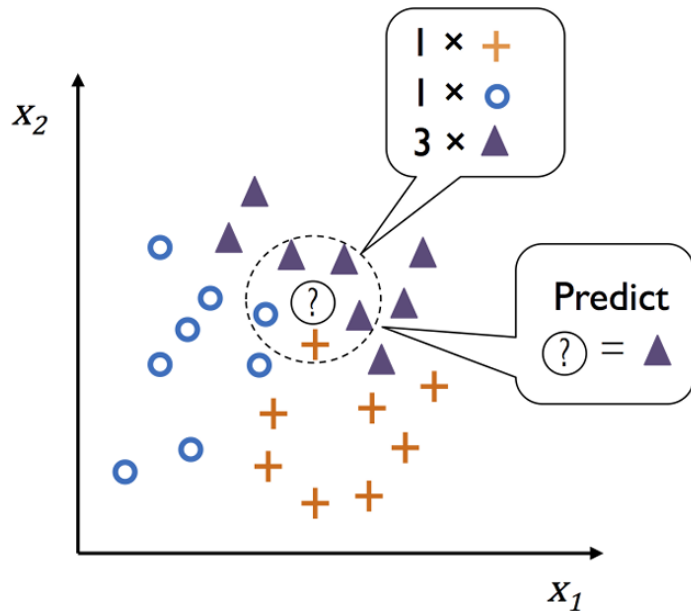
- Calculate distance
- Get neighbors

Mais la **prédiction** changera ! Au lieu de la calcule de la moyenne, on utilise le vote.

- Predict



Prédiction – Vote (1/2)



Label de test :

- Le nombre de k plus proches voisins pour chaque classe est compté (appelé **vote**)
- La classe avec le nombre maximal de votes est choisie

Probabilité du label de test :

- La proportion des k plus proches voisins appartenant à la classe prédite

$$P(y_0 = k | X = x_0) = \frac{1}{K} \sum_{x_i \in N_0} I(y_i = k)$$

Prédiction – Vote (2/2)

- En cas de classification binaire → **Majority voting** (*classe avec une probabilité >0.50*)

y: ● ● ● ● ■ ■ ■ ■ ■ ■ ■ ■

k pair ?

Majority vote: ■

Plurality vote: ■

- En cas de classification multi-classe → **Plurality voting** (*pas besoin d'avoir une probabilité >0.50 , mais celle avec la probabilité maximale*)

y: ● ● ● ■ ■ ■ ◆ ◆ ◆ ◆

Majority vote: None

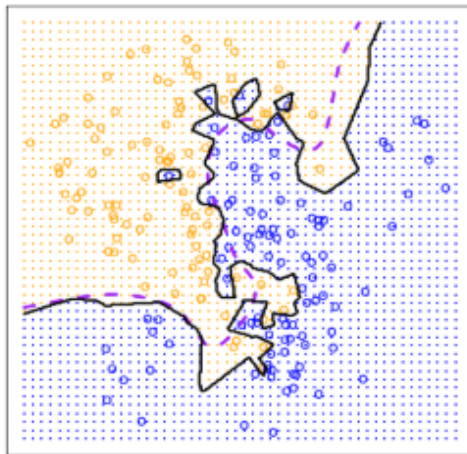
Plurality vote: ◆

Nombre de voisin

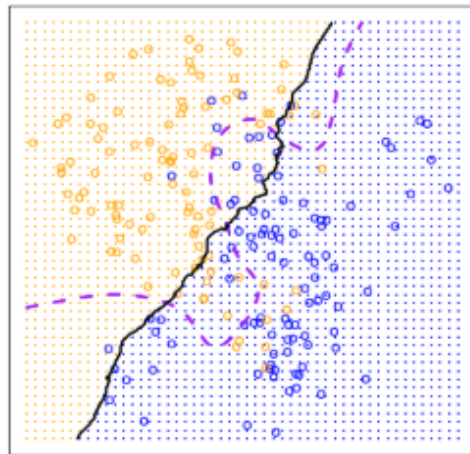
En **classification**, on choisit souvent **k impair** (1,3,5,...) pour éviter les égalités lors du vote majoritaire

(on n'a pas cette contrainte en régression)

Petit k :
Overfitting



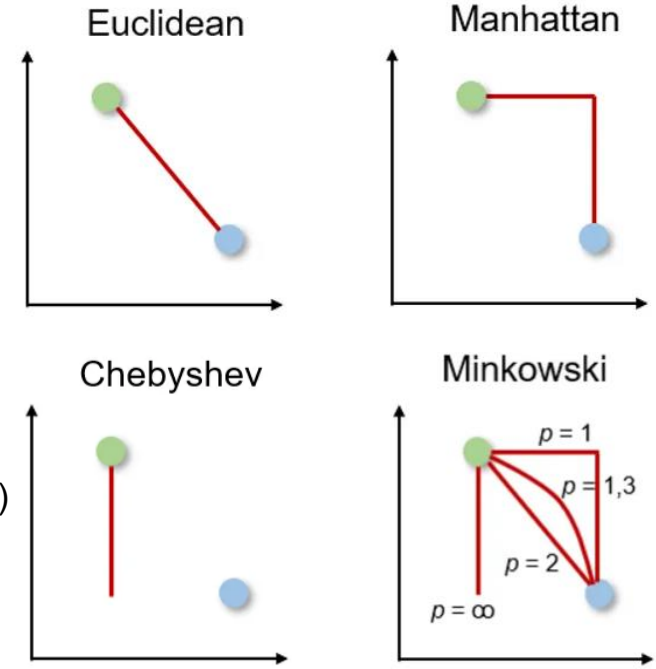
Grand k :
Underfitting



Métriques de Distance (1/3)

Principalement pour données numériques continues :

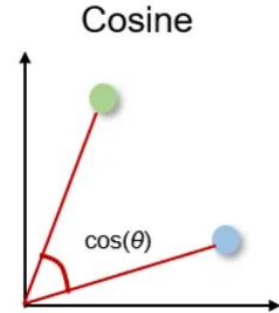
- **Euclidienne**
 - Distance “à vol d’oiseau”, pénalise les grandes distances
- **Manhattan**
 - Distance parcourue dans une grille (ex: rues en ville)
- **Chebyshev**
 - On regarde uniquement la plus grande différence
 - Mouvement dominé par l’axe le plus “éloigné” (ex: robotique)
- **Minkowski** (Distance de généralisation)
 - $p=1$ Manhattan, $p=2$ Euclidean, $p=\infty$ Chebyshev



Métriques de Distance (2/3)

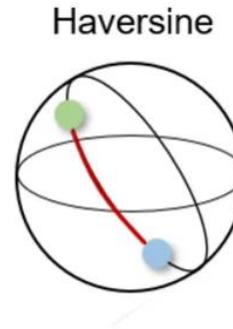
- Cosine

- Mesure l'angle entre deux vecteurs
- Se concentre sur la direction
- Données textes (ex: 'génial' et 'nul')



- Haversine

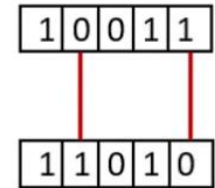
- Terre sphérique (pas ligne droite)
- Réalité géographique (ex: latitude, longitude)



- Hamming

- Nombre de positions différentes entre deux chaînes
- Chaînes binaires (ex: ADN)

Hamming



Métriques de Distance (3/3)

- **Jaccard**

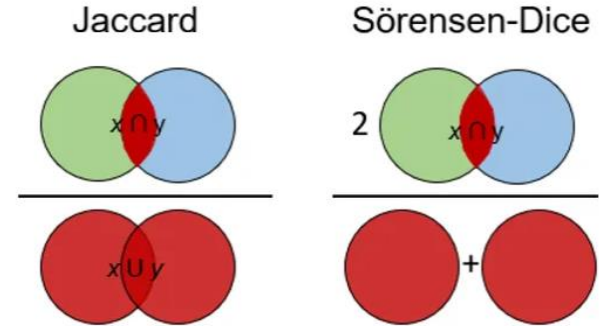
- Compare uniquement la présence/absence d'éléments
- Ignore les doublons
- Données binaires, textes

- **Sørensen-Dice**

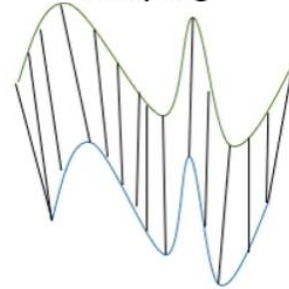
- Donne plus de poids aux éléments communs
- Analyse d'images (segmentation)

- **Dynamic Time Warping**

- Séries temporelles avec décalage temporel
- Signaux physiologiques, reconnaissance vocale



Dynamic Time
Warping



Système de Recommandation avec kNN

- kNN permet de recommander des items en trouvant les utilisateurs ou objets les plus similaires



Amazon et kNN : Amazon a utilisé pendant longtemps des méthodes **proches du kNN** (item-based)

- On mesure la similarité entre produits
- On estime une probabilité d'achat



Netflix et kNN : Recommander des films en utilisant les préférences d'utilisateurs similaires (user-based)

- Tu aimes :
 - Inception ★★★★★
 - Interstellar ★★★★★
- Tes voisins aiment : ★★★★★
 - The Matrix
- Netflix recommande **The Matrix**

La **recommandation** est proche de la régression (scores continus), mais son objectif final est le **ranking**

(Donc, un autre type de problèmes en apprentissage supervisé)

Classification

kNN : Exemple

- Quelle app utilisez vous le plus fréquemment ?
- Combien d'heures par jour ?
- Combien de notifications par jour ?
- Combien d'amis utilisent la même app ?
- C'est un jeu ou autre chose ?
- Vous pensez être addict ?



Dataset : Addict d'app

Person	Daily usage (hrs)	Notification s/day	Friends using app	Games (0/1)	Class
A	5	40	8	1	1
B	1	5	2	0	0
C	3	20	5	1	1
D	0,5	2	1	0	0
X	2	50	3	0	?

Classification

Evaluation de performance

Matrice de Confusion



Actual	Predicted	
	TN	FP
	FN	TP

TN : True Negative

FN : False Negative

TP : True Positive

FP : False Positive

On cherche la proportion de classes prédites par rapport aux classes réelles

Cas d'Usage

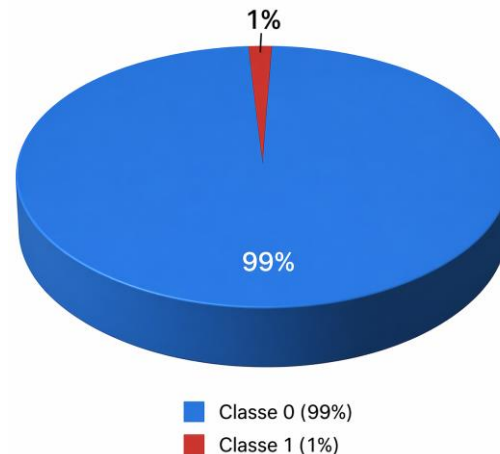
Détection cancer :

- 1% des patients malades (Classe 1)

Si ton modèle dit 'tout le monde est sain'

Accuracy ?

Accuracy : 99 %



Mais c'est un bon indicateur pour ce problème ? Quoi d'autre ?

Métriques de performance (1/2)

En cas de classification **binaire**

- **Accuracy** : pourcentage de prédictions **correctes**
 - **TN** ↑ **TP** ↑
- **Précision** : **qualité** des prédictions **positives**
 - **TP** ↑ **FP** ↓
- **Recall (Rappel)** : **capacité** à détecter les **positifs**
 - **TP** ↑ **FN** ↓
- **F1-score** : Moyenne harmonique entre **précision** et **rappel**
 - **TP** ↑ **Compromis** entre **FP** et **FN**
 - Utile quand les classes sont déséquilibrées

	Predicted	
	TN	FP
Actual	FN	TP

$$Accuracy = \frac{TN + TP}{TN + FN + FP + TP}$$

$$Precision = \frac{TP}{FP + TP}$$

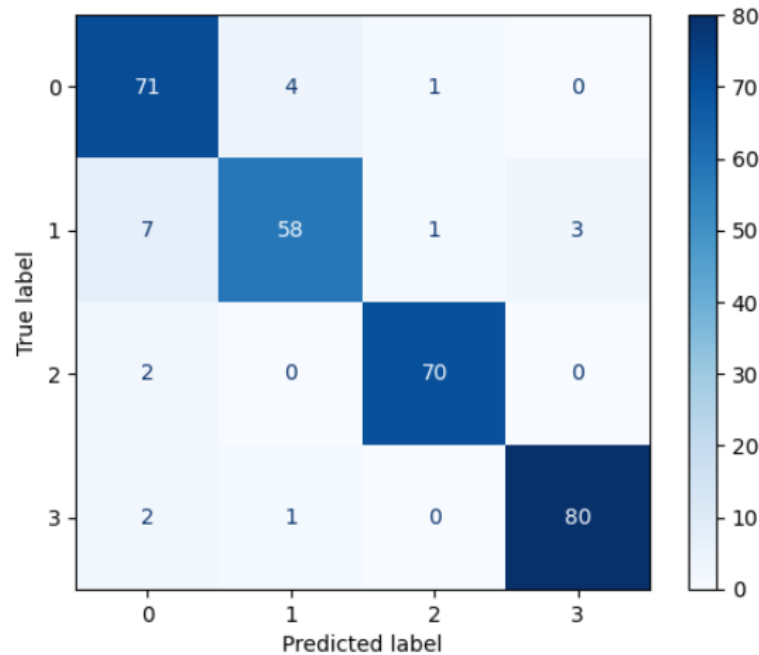
$$Recall = \frac{TP}{FN + TP}$$

$$F1score = \frac{2 * Precision * Recall}{Precision + Recall}$$

Métriques de performance (2/2)

En cas de classification **multi-classe**

- **Accuracy**
 - Calculée toujours globalement
 - **Précision & Recall (Rappel) & F1-score**
 - Calculés **par classe**
- Puis agrégés de manière :
- **macro average** : moyen simple (toutes les classes même poids)
 - **weighted average** : moyen pondéré par support (nombre d'exemples par classe)



Python

```
from sklearn.metrics import confusion_matrix
from sklearn.metrics import accuracy_score
from sklearn.metrics import precision_score
from sklearn.metrics import recall_score
from sklearn.metrics import f1_score
```

Utilisation :

`confusion_matrix(y_test, y_pred)`

`accuracy_score(y_test, y_pred)`

`precision_score(y_test, y_pred)`

`recall_score(y_test, y_pred)`

`f1_score(y_test, y_pred)`

En cas de classification **multi-classe** :

`precision_score(y_test, y_pred, average='binary')`

- 'binary' → pour classification binaire (par défaut)
- 'macro' → pondération égale pour chaque classe
- 'weighted' → pondéré par le support

(pareil pour `recall_score` et `f1_score`)

OU

```
from sklearn.metrics import classification_report
```

`classification_report(y_test, y_pred)`

	precision	recall	f1-score	support
0	0.87	0.93	0.90	76
1	0.92	0.84	0.88	69
2	0.97	0.97	0.97	72
3	0.96	0.96	0.96	83
accuracy			0.93	300
macro avg	0.93	0.93	0.93	300
weighted avg	0.93	0.93	0.93	300

Quiz



Une entreprise développe un modèle pour filtrer des emails spam.

- Lorsqu'un email est classé comme spam, il est directement supprimé
- Il est très important de ne pas supprimer des emails légitimes
- On accepte que certains spams passent

Quelle métrique est la plus appropriée pour évaluer le modèle ?

- a) Accuracy
- b) Précision
- c) Rappel (Recall)
- d) F1-score

Courbe ROC (**R**eceiver **O**perating **C**haracteristic)

Relation entre **FPR** (FP rate) vs **TPR** (TP rate)

Variation du seuil :

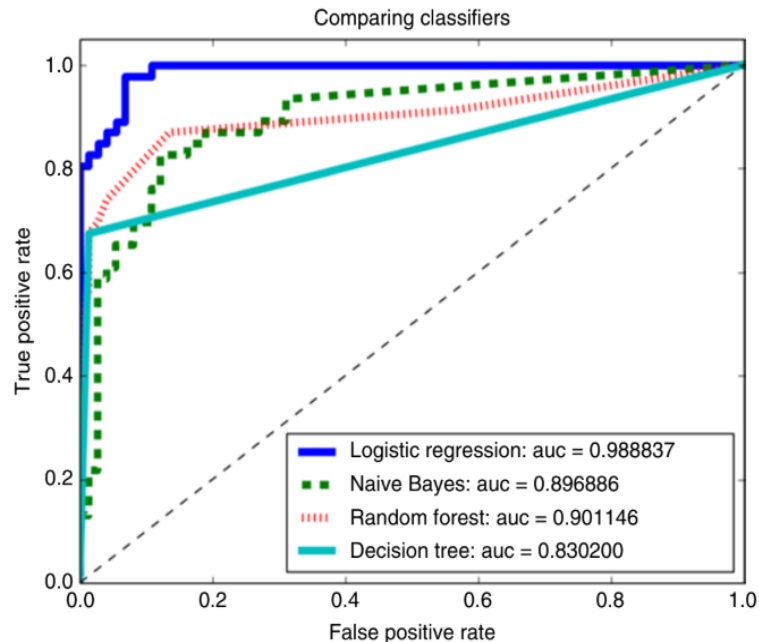
- Selon différents seuils de décision sur les probabilités
- Chaque seuil donne un couple (FPR, TPR)

```
from sklearn.metrics import roc_curve
```

```
fpr, tpr, thresholds = roc_curve(y_test, y_pred)
```

Interprétation :

- Le coin supérieur gauche (0.0, 1.0) correspond à un **classifieur parfait**
- Plus la courbe est proche de ce coin, meilleur est le modèle
- Pas toujours facile à interpréter
- Il faut donc une métrique numérique !



AUC (Area Under the ROC Curve)

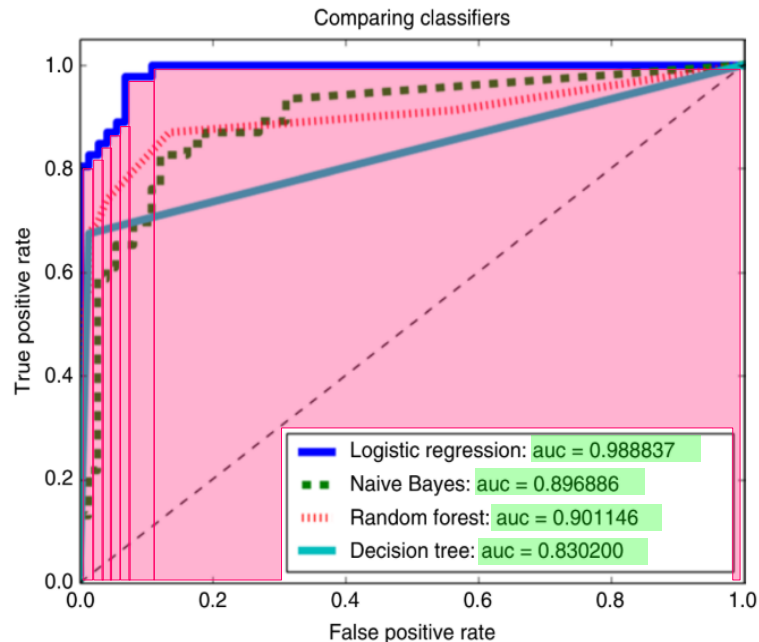
Aire sous la courbe ROC (AUC)

- On somme les aires des trapèzes entre chaque point

```
from sklearn.metrics import auc, roc_auc_score  
fpr, tpr, thresholds = roc_curve(y_test, y_pred)  
auc = auc(fpr, tpr)  
OU  
auc = roc_auc_score(y_test, y_pred)
```

Interprétation :

- Si plus proche de 1, meilleur modèle
- Si 0.5, modèle aléatoire

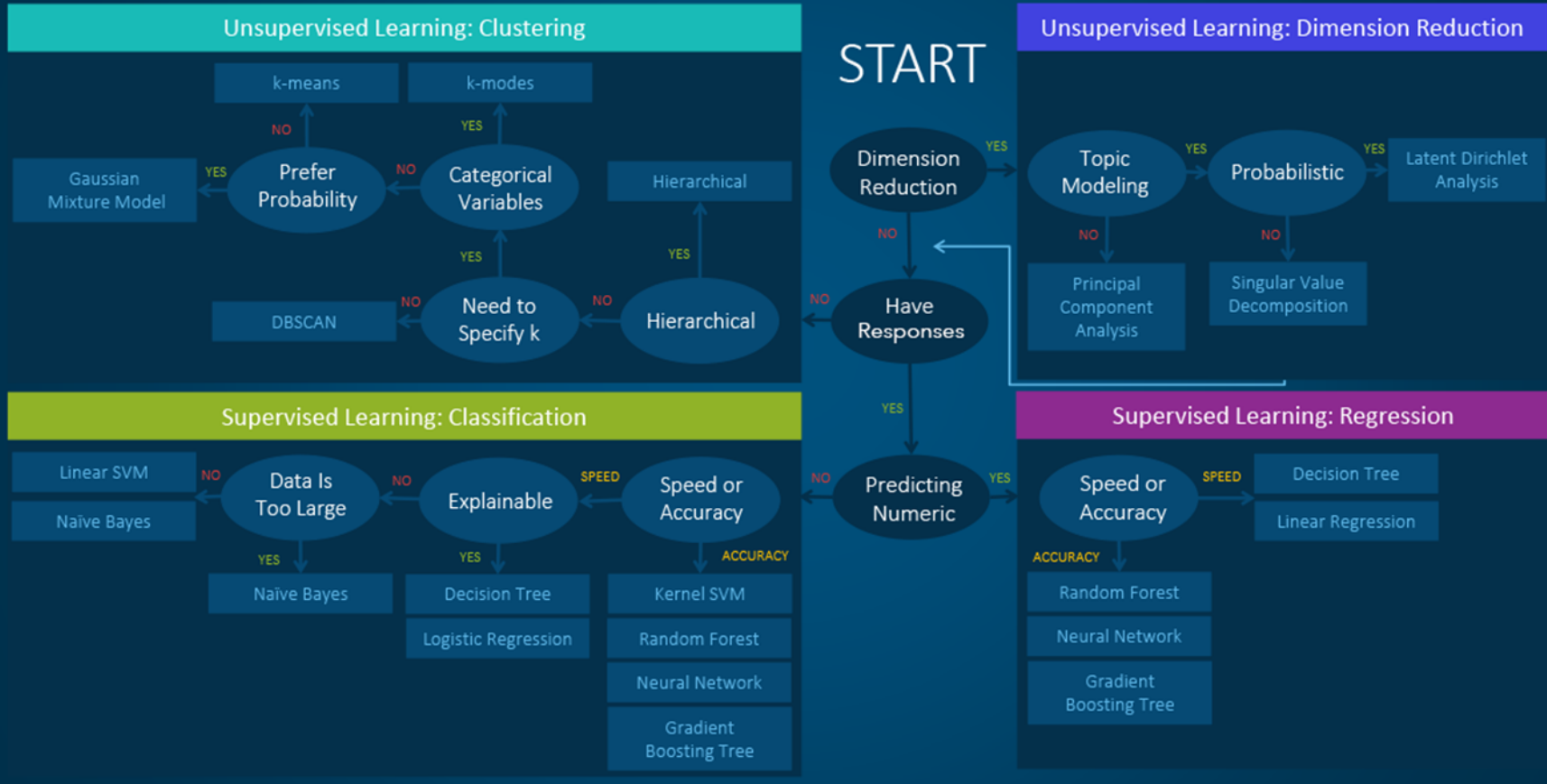


Classification

Naïve Bayes

Cette table doit être prise comme un guide mais n'est pas toujours vraie : pour certains cas, le choix peut être différent.

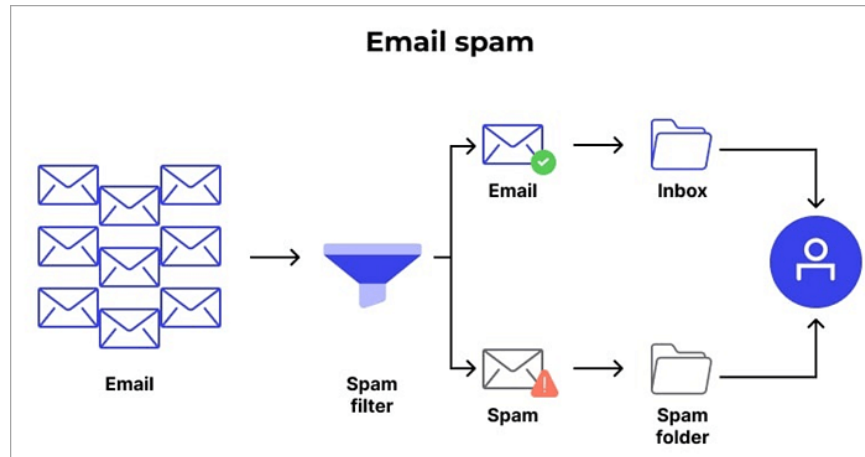
Machine Learning Algorithms Cheat Sheet



Cas d'usage

Détection du spam :

Un email est *spam* ou *non-spam*



Certains mots apparaissent plus souvent dans les spams

- **Comment vous feriez ?**

- “compter les mots”
- “regarder fréquence”



Cas d'usage

Détection du spam :

Un email est *spam* ou *non-spam*

Dans ce contexte ;

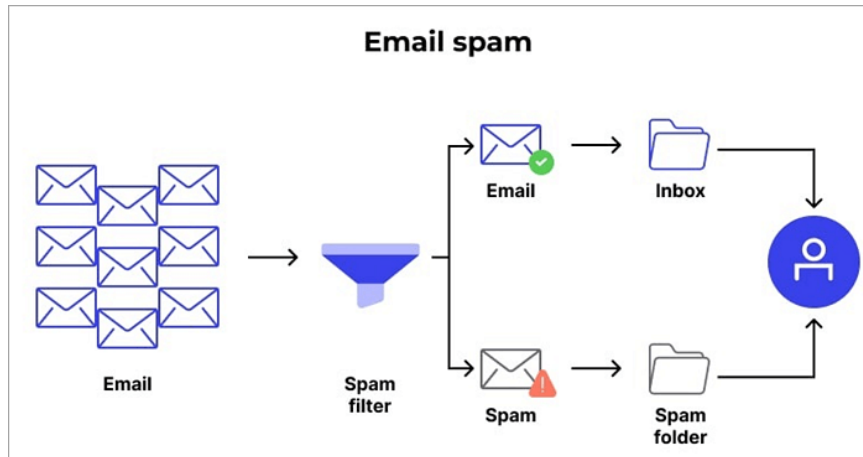
- **kNN** est généralement moins adapté car distance non fiable dans les espaces de grande dimension
- Tandis que le modèle **Naïve Bayes** fonctionne bien en calculant les probabilités

Pour chaque mot (ex : "free"), on calcule :

- fréquence dans les spams
- fréquence dans les non-spams

$$P(\text{Spam} \mid \text{mots}) = \frac{P(\text{mots} \mid \text{Spam}) \cdot P(\text{Spam})}{P(\text{mots})}$$

Si un mail contient beaucoup de mots typiques du spam, la probabilité augmente



Principe de Naïve Bayes

Basée sur la **théorème de Bayes** : $P(y|X) = \frac{P(X|y) \cdot P(y)}{P(X)}$

Probas :

- $P(y|X)$: la **postérieure** (ce que l'on cherche à estimer)
- $P(X|y)$: la **vraisemblance** (à quel point les données sont probables sachant la classe)
- $P(y)$: le **priori** (probabilité a priori de la classe dans les données d'entraînement)
- $P(X)$: l'**évidence** (constante de normalisation, souvent ignorée en classification)

Pourquoi « Naïve » ?

- Car Naïve Bayes suppose repose sur une **hypothèse simplifiée** :
 - Les **features** sont supposées **indépendantes**
 - Cette hypothèse permet de simplifier le calcul de la **vraisemblance** dans un problème multivarié :

$$P(X|y) = P(x_1|y) \cdot P(x_2|y) \cdot \dots \cdot P(x_n|y) = \prod_{i=1}^n P(x_i|y)$$

Procédure – Training

$P(y)$: On calcule la proportion de chaque classe (y) dans le dataset (probabilité a **priori**)

$P(X)$: L'évidence peut s'écrire comme :

$$P(X) = \sum_{y \in \mathcal{Y}} P(X | y) P(y)$$

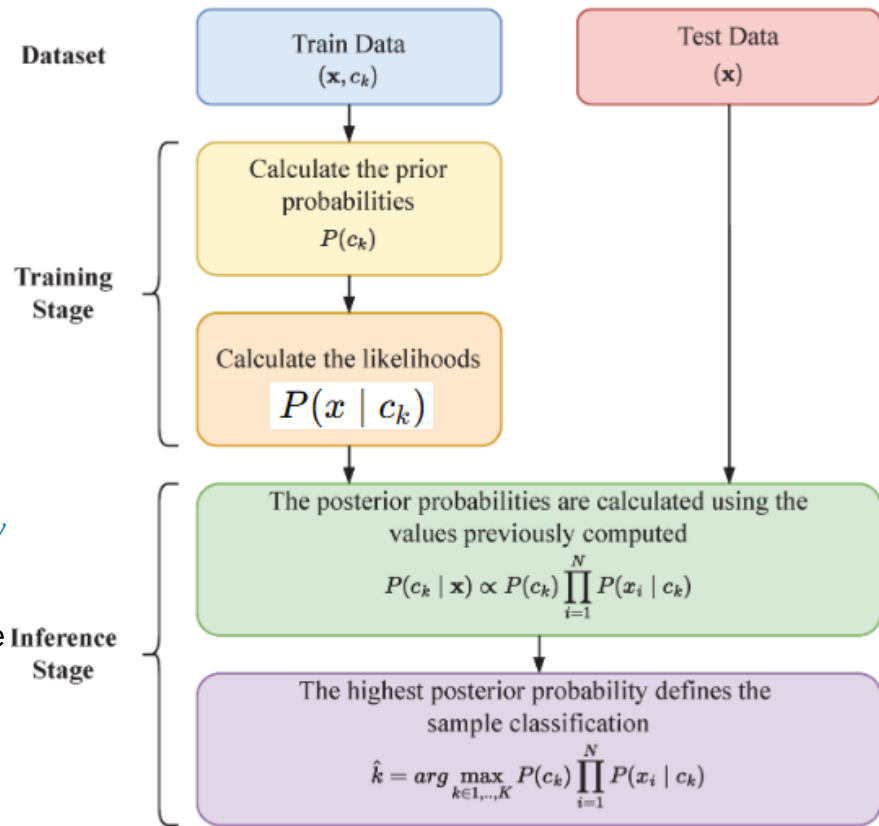
Donc, ce suffit de calculer les vraisemblances $P(X|y)$ pour chaque y

$P(X | y)$: À quel point les données X sont probables si la classe est y

Hypothèse naïve : On suppose que les features sont indépendantes

Pour chaque feature, on calcule la **vraisemblance** et on les multiplie

$$P(X | y) = \prod_{i=1}^n P(x_i | y)$$



Procédure – Prédiction

$P(y | X)$: (probabilité a **posteriori**)

$$P(y | X) = \frac{P(y) \prod_{i=1}^n P(x_i | y)}{\sum_c P(c) \prod_{i=1}^n P(x_i | c)}$$

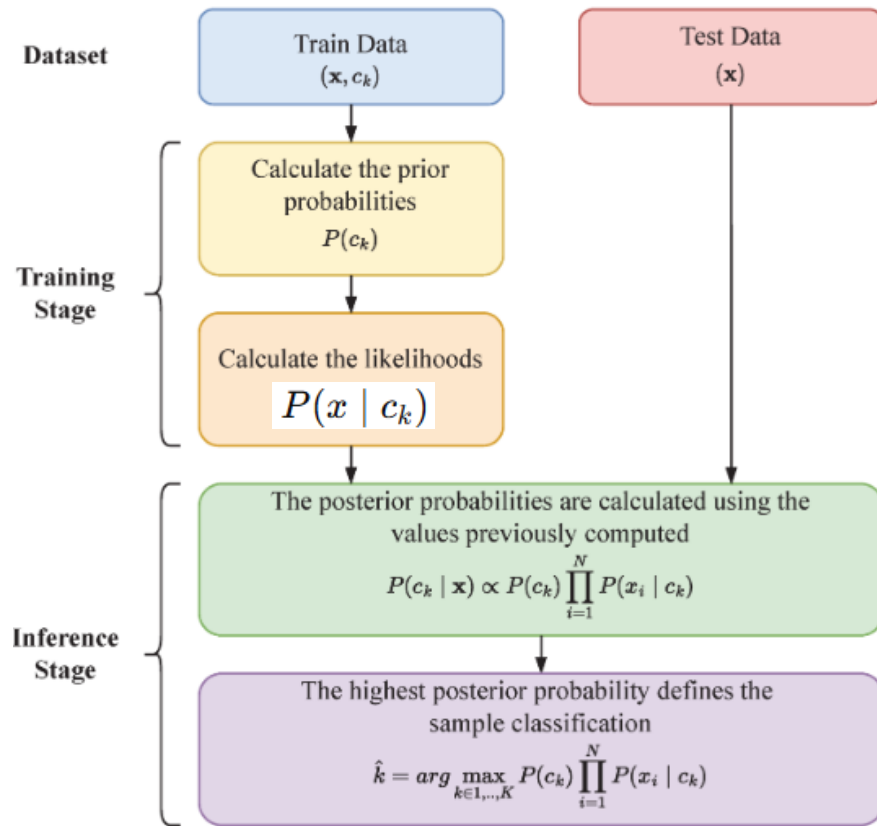
Comme le dénominateur est le même pour toutes les classes, on peut simplement utiliser le numérateur.

$$P(y|X) \propto P(y) \prod_{i=1}^n P(x_i|y)$$

Mais pour le calcul de la vraie probabilité, on en aura besoin.

Prédiction :

$$\hat{y} = \arg \max_y \left(P(y) \prod_{i=1}^n P(x_i|y) \right)$$



Procédure – Prédiction (Log)

$P(y | X)$: (probabilité a **posteriori**)

- Les probabilités sont souvent très petites
- Les multiplier peut provoquer des erreurs numériques
- On peut donc transformer la multiplication en somme avec la transformation logarithmique

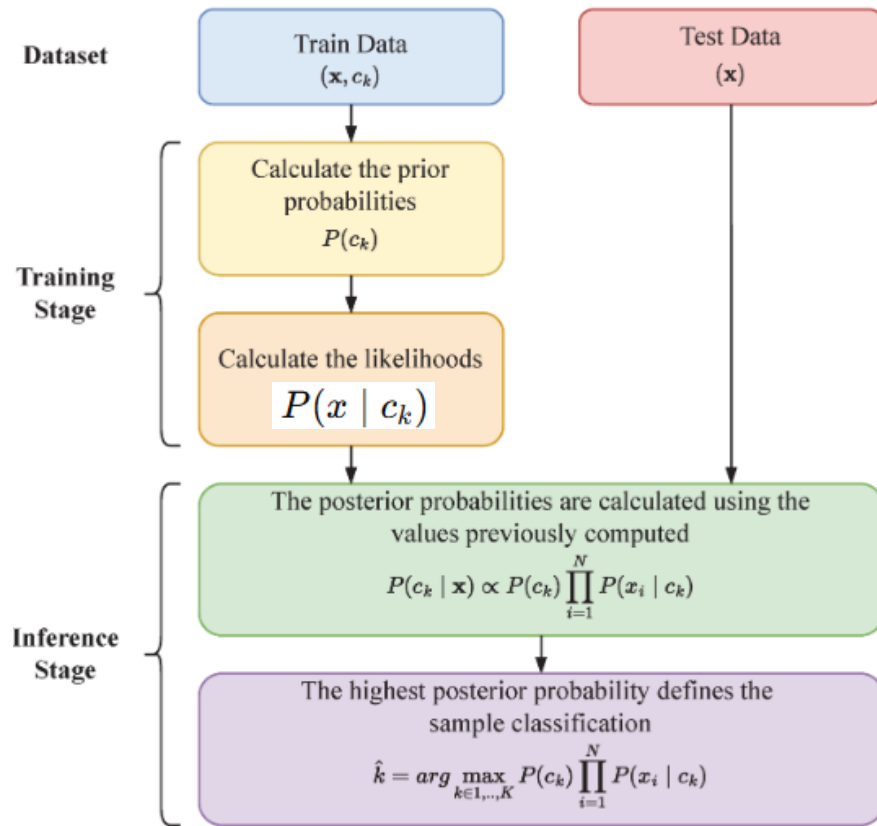
$$P(y|X) \propto P(y) \prod_{i=1}^n P(x_i|y)$$



$$\log P(y|X) \propto \log P(y) + \sum_{i=1}^n \log P(x_i|y)$$

Prédiction :

$$\hat{y} = \arg \max_y \left(\log P(y) + \sum_{i=1}^n \log P(x_i|y) \right)$$



Procédure

- **Training :**
 - Estimer les probabilités a **priori** $P(y)$ et les **vraisemblances** $P(X|y)$
- **Prédiction:**
 - Appliquer la règle de Bayes et choisir la classe ayant la plus grande probabilité a **posteriori** $P(y|X)$
- **L'entraînement et la prédiction sont rapides à calculer**
(pas d'optimisation, pas de calcul de distance, pas de descente de gradient)

Classification

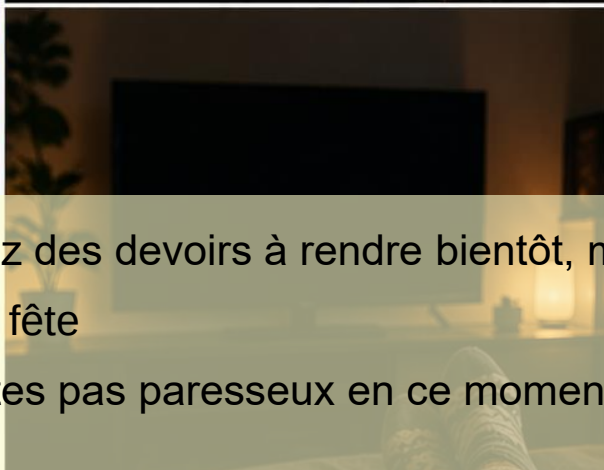
Naïve Bayes : Exemple



Que faire ce soir ?



Vous avez des devoirs à rendre bientôt, mais rien d'urgent
Il y a une fête
Vous n'êtes pas paresseux en ce moment



Dataset : Décision d'activité

Le devoir, la fête et la paresse sont indépendants ?

Hypothèse d'indépendance entre les features ✓

Les données correspondent à un ensemble d'exemples passés des derniers jours

Features :

- Deadline? : {Urgent, Near, None}
- Is there a party? : {Yes, No}
- Lazy? : {Yes, No}

Classes (activités) : {Party, Study, Pub, TV}

- 4 classes exclusives

Deadline?	Is there a party?	Lazy?	Activity
Urgent	Yes	Yes	Study
Urgent	No	Yes	Study
Urgent	No	No	Study
Near	Yes	Yes	Party
Near	No	No	Study
Near	No	Yes	TV
Near	Yes	Yes	Party
None	Yes	No	Party
None	No	No	Pub
None	Yes	No	Party

Near

Yes

No

?

On veut calculer la probabilité de chaque activité étant donné ces valeurs de features :

$P(y \mid \text{near, party, not lazy})$?

Probabilité à priori

P(y)

10 expériences dans l'espace

- $P(\text{Party}) = \frac{4}{10}$
- $P(\text{Study}) = \frac{4}{10}$
- $P(\text{Pub}) = \frac{1}{10}$
- $P(\text{TV}) = \frac{1}{10}$

Deadline?	Is there a party?	Lazy?	Activity
Urgent	Yes	Yes	Study
Urgent	No	Yes	Study
Urgent	No	No	Study
Near	Yes	Yes	Party
Near	No	No	Study
Near	No	Yes	TV
Near	Yes	Yes	Party
None	Yes	No	Party
None	No	No	Pub
None	Yes	No	Party

Vraisemblance

$$P(\text{near, party, not lazy} \mid y) = P(\text{near} \mid y) \times P(\text{party} \mid y) \times P(\text{not lazy} \mid y)$$

$$\blacksquare P(\text{near} \mid \text{Party}) \times P(\text{party} \mid \text{Party}) \times P(\text{not lazy} \mid \text{Party})$$

$$\blacksquare = \frac{2}{4} * \frac{4}{4} * \frac{2}{4}$$

$$\blacksquare P(\text{near} \mid \text{Study}) \times P(\text{party} \mid \text{Study}) \times P(\text{not lazy} \mid \text{Study})$$

$$\blacksquare = \frac{1}{4} * \frac{1}{4} * \frac{2}{4}$$

$$\blacksquare P(\text{near} \mid \text{Pub}) \times P(\text{party} \mid \text{Pub}) \times P(\text{not lazy} \mid \text{Pub})$$

$$\blacksquare = \frac{0}{1} * \frac{0}{1} * \frac{1}{1}$$

$$\blacksquare P(\text{near} \mid \text{TV}) \times P(\text{party} \mid \text{TV}) \times P(\text{not lazy} \mid \text{TV})$$

$$\blacksquare = \frac{1}{1} * \frac{0}{1} * \frac{0}{1}$$

Deadline?	Is there a party?	Lazy?	Activity
Urgent	Yes	Yes	Study
Urgent	No	Yes	Study
Urgent	No	No	Study
Near	Yes	Yes	Party
Near	No	No	Study
Near	No	Yes	TV
Near	Yes	Yes	Party
None	Yes	No	Party
None	No	No	Pub
None	Yes	No	Party

Priori * Vraisemblance

P(y) x P(near, party, not lazy | y)

$$P(\text{Party}) \times P(\text{near, party, not lazy} \mid \text{Party}) = \frac{4}{10} * \left(\frac{2}{4} * \frac{4}{4} * \frac{2}{4} \right) = 1/10$$

$$P(\text{Study}) \times P(\text{near, party, not lazy} \mid \text{Study}) = \frac{4}{10} * \left(\frac{1}{4} * \frac{1}{4} * \frac{2}{4} \right) = 1/80$$

$$P(\text{Pub}) \times P(\text{near, party, not lazy} \mid \text{Pub}) = \frac{1}{10} * \left(\frac{0}{1} * \frac{0}{1} * \frac{1}{1} \right) = 0$$

$$P(\text{TV}) \times P(\text{near, party, not lazy} \mid \text{TV}) = \frac{1}{10} * \left(\frac{1}{1} * \frac{0}{1} * \frac{0}{1} \right) = 0$$

On peut déjà voir le gagnant, car $P(y|X) \propto P(y) \prod_{i=1}^n P(x_i|y)$. Mais on va calculer la vraie probabilité !

Probabilité à posteriori

$$P(y \mid \text{near, party, not lazy}) = \frac{P(y) \times P(\text{near, party, not lazy} \mid y)}{\sum_y P(y) \times P(\text{near, party, not lazy} \mid y)}$$

$$P(\text{Party} \mid \text{near, party, not lazy}) = \frac{1}{10} / \left(\frac{1}{10} + \frac{1}{80} + 0 + 0 \right) = 0.89$$

$$P(\text{Study} \mid \text{near, party, not lazy}) = \frac{1}{80} / \left(\frac{1}{10} + \frac{1}{80} + 0 + 0 \right) = 0.11$$

$$P(\text{Pub} \mid \text{near, party, not lazy}) = 0 / \left(\frac{1}{10} + \frac{1}{80} + 0 + 0 \right) = 0$$

$$P(\text{TV} \mid \text{near, party, not lazy}) = 0 / \left(\frac{1}{10} + \frac{1}{80} + 0 + 0 \right) = 0$$

Selon le modèle Naïve Bayes, vous irez à la fête ce soir !



Lissage – Smoothing parameter

{Party, Study, Pub, TV} = {0.89, 0.11, 0, 0}

Pour éviter les **probabilités nulles**, on peut appliquer un lissage aux calculs.

- Car en Naive Bayes, une seule probabilité nulle suffit à annuler tout le calcul, en raison de l'hypothèse d'indépendance qui implique la multiplication de toutes les probabilités.
- **Sans lissage**, on calcule par exemple $P(\text{near} \mid \text{Party})$ en comptant le nombre d'occurrences et en divisant par le nombre total d'échantillons

$$P(x_i = v \mid y) = \frac{\text{count}(v, y)}{\text{count}(y)}$$

- **Avec lissage**, on ajoute une constante de lissage (α : smoothing constant)

$$P(x_i = v \mid y) = \frac{\text{count}(v, y) + \alpha}{\text{count}(y) + \alpha K}$$

Ex : $\alpha = 1$, K pour feature 'Deadline' = 3 (3 choix possible), K pour 'Is there a party' = 2, K pour 'Lazy' = 2

$$P(\text{near, party, not lazy} \mid \text{Pub}) = \frac{0+1}{1+3} * \frac{0+1}{1+2} * \frac{1+1}{1+2} = \frac{1}{4} * \frac{1}{3} * \frac{2}{3} \quad (\text{pas } 0)$$

Classification

Naïve Bayes : Variantes

Variantes de Naïve Bayes

- ❑ **Categorical Naïve Bayes**

- ❑ Pour les variables **catégorielles discrètes** : plusieurs catégories possibles par feature (ex : rouge/vert/bleu, urgent/near/none (deadline))

- ❑ **Bernoulli Naïve Bayes**

- ❑ Pour les variables **binaires** : 0/1 → présence ou absence d'une feature (ex : mot présent/absent)

- ❑ **Multinomial Naïve Bayes**

- ❑ Pour les variables discrètes représentant des **comptages** (ex : fréquence de mots dans un texte)
 - ❑ **Complement Naïve Bayes** : Variante de Multinomial NB, particulièrement adaptée aux datasets déséquilibrés

- ❑ **Gaussian Naïve Bayes**

- ❑ Pour les variables **continues** suivant une distribution **gaussienne** (ex : hauteur (m), température (°C))

Vraisemblance de chaque feature devient :

$$P(x_i | y) = \frac{1}{\sqrt{2\pi\sigma_y^2}} \exp\left(-\frac{(x_i - \mu_y)^2}{2\sigma_y^2}\right)$$

Comparaison de variantes de Naïve Bayes

Variante de NB	Type de Feature	Conversion	Cas d'usage
Categorical	Catégorielle	Encodage en catégories	Variables qualitatives (couleur, type de produit, etc.)
Bernoulli	Binaire	Binarisation 0/1	Présence de mot-clé
Multinomial	Comptage, fréquences	Texte → comptage	NLP / classification de texte, détection spam
Gaussian	Continue	Non, mais suppose une distribution gaussienne	Données de capteurs, mesures physiques

On peut perdre de la précision des features en convertissant des données !

Dans le cas de **données mixtes** (différents types de features) :

- Une combinaison de différentes variantes est possible
- Ou bien convertir les features vers un seul type

Mais **en pratique, NB est souvent trop simpliste pour des données mixtes !**

Avantages vs Limites de Naïve Bayes

Avantages

- **Simple** à comprendre au niveau conceptuel
- **Très rapide** en entraînement et en prédiction
- Fonctionne bien avec les features **catégorielles** (et aussi **textuelles**)
- Performant même avec **peu de données**
- Pas de mise à l'échelle

Limites

- **Hypothèse forte d'indépendance entre les features**
- Sensible à la qualité des probabilités estimées (données rares)
- Peut être **moins performant** que des modèles plus complexes sur des **données riches en dépendances**
- **Peu flexible**
- **Conversion** des features (perd de la précision)
- Principalement utilisé pour des tâches de **classification**

Python

```
from sklearn.naive_bayes import CategoricalNB
from sklearn.naive_bayes import BernoulliNB
from sklearn.naive_bayes import MultinomialNB
from sklearn.naive_bayes import ComplementNB
from sklearn.naive_bayes import GaussianNB
```

Hyperparamètres

- `alpha` : lissage des probabilités (pour tous les types sauf GaussianNB)
- `var_smoothing` : stabilisation des variances (pour seulement GaussianNB)

Très peu d'hyperparamètres en Naïve Bayes
